Static code analysis for security audit

Concepts and Techniques

Tosin Daniel Oyetoyan

17th April 2026

Agenda

- Weaknesses and vulnerabilities
- Example of Security vulnerability
- Code vulnerabilities (Good vs. Bad)
- What and why static code analysis
- Techniques used in sast tools
- Performance metrics
- LLM-Assisted SAST

Weaknesses and Vulnerabilities

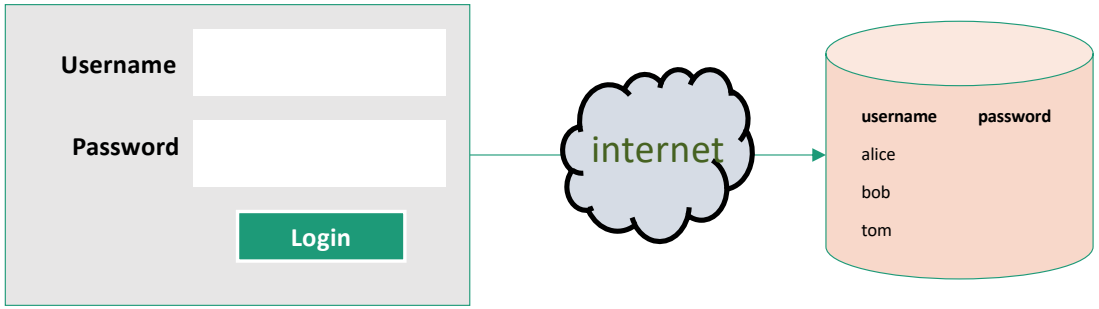
- Software weaknesses are errors in software implementation, code, design, or architecture that if left unaddressed could result in systems and networks being vulnerable to attack (CWE)
  - Examples include: buffer overflows, format strings, etc., structure and validity problems, code evaluation and injection, authentication errors, resource management errors, insufficient verification of data
- A software vulnerability, such as those enumerated on the Common Vulnerabilities and Exposures (CVE) List, is a mistake/weakness in software that can be <u>directly</u> used by an attacker to gain access to a system or network
- An Exploit is a piece of software <u>containing attack vectors</u> that could be <u>directly</u> used to take advantage of a vulnerability in a system (see Metasploit, exploit-db)

Code quality vs security

Security: Confidentiality, Integrity, Availability, Authentication, Authorization, and Non-Repudiation

Security vulnerability - SQL Injection (Example)

- SQL Injection occurs when untrusted data are added to database queries to change the application's design
- Dynamic string building when programmers mix query language fragments and untrusted data

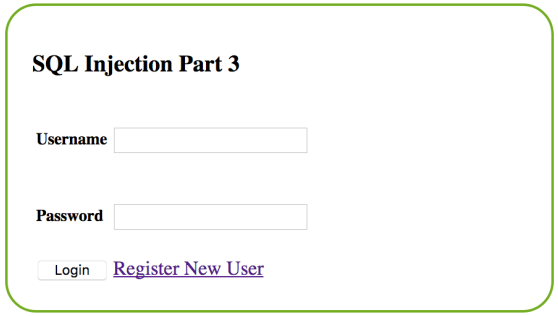



```
String query = "SELECT * FROM users WHERE username = " + request.getParameter("user_id") + "AND password = "+  request.getParameter("password") ;
```

The injection vector can be passed via the ***request.getParameter()***!!!

SQLi – Subverting application logic

- An attacker can bypass an authentication system



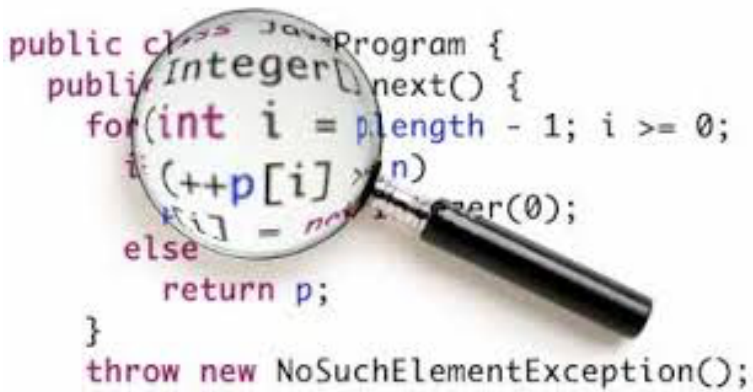

```
public String authenticateWithSalt(String userid, String password) {
     String query0 = "SELECT salt FROM owasp_users WHERE username = '"+userid+"'";
     String salt = getSalt(query0);
     String passhash = null;
     try {
          byte[] saltbytes = DatatypeConverter.parseHexBinary(salt);
          PasswordHash ph = new PasswordHash(PasswordHash.SHA256);
          passhash = ph.generateHashWithSalt(password, saltbytes);
     } catch (NoSuchAlgorithmException e) {
          //e.printStackTrace();
     }
     String query = "SELECT * FROM owasp_users WHERE username = '"+userid+"' AND
                    password = '"+passhash+"'";
```

username = *"test1' or 1 = 2-- "*

What is Static Analysis?

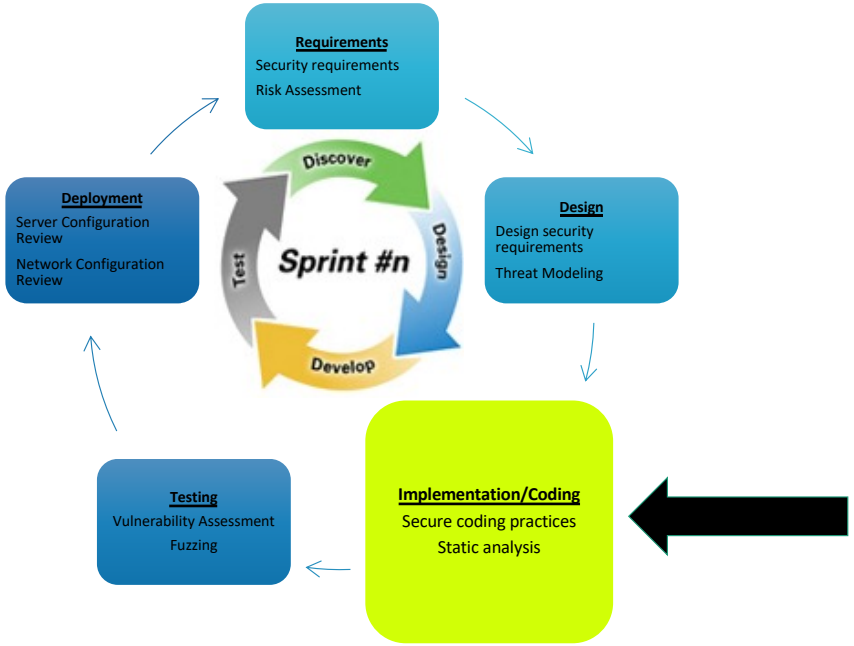
- Passive scanning of application code without executing it
- a *source code security analyzer*
  - examines source code to
  - detect and report weaknesses that can lead to security vulnerabilities

- Results can be in a report form
- May be integrated into IDEs
- May be integrated into CI/CD environments



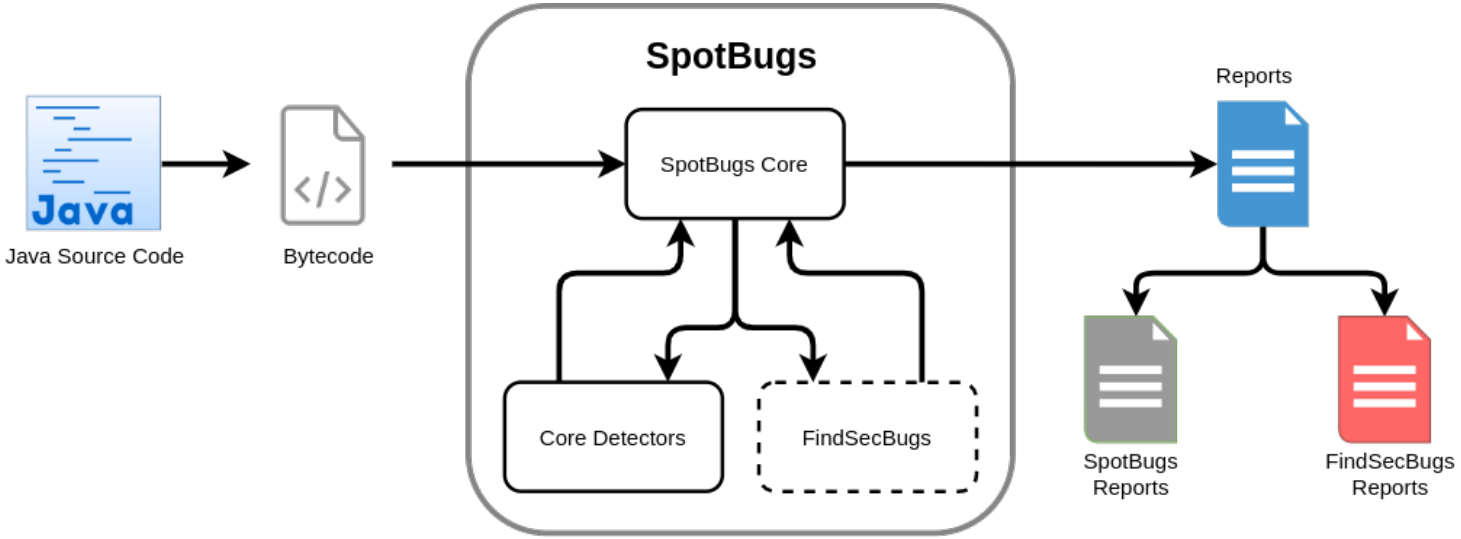
Why Static Code Analysis for security audits?

- Can catch security defects early in the SDLC
- Significant aid for code review (Starting point)



Static Application Security Tools (SAST)

- ✓ FindSecBugs
- ✓ SpotBugs
- ✓ CheckMarx
- ✓ Fortify
- ✓ …



Code vulnerabilities (Good vs. Bad)

- Benchmark - NIST testsuite
- XSS - example

```java
/* goodG2B() - uses goodsource and badsink */
private void goodG2B(HttpServletRequest request, HttpServletResponse response)
        throws Throwable
{
    String data;

    /* FIX: Use a hardcoded string */
    data = "foo";

    if (data != null)
    {
        /* POTENTIAL FLAW: Display of data in web page
         *  after using replaceAll() to remove script
         * tags, which will still allow XSS with strings
         * like <scr<script>ipt> (CWE 182: Collapse
         * of Data into Unsafe Value)
         */
        response.getWriter().println("<br>bad(): data = "
         + data.replaceAll("(<script>)", ""));
    }

}
```

GoodSource-BadSink

```java
/* uses badsource and badsink */
public void bad(HttpServletRequest request, HttpServletResponse response)
        throws Throwable
{
    String data;

    /* POTENTIAL FLAW: Read data from a
     * querystring using getParameter
     */
    data = request.getParameter("name");

    if (data != null)
    {
        /* POTENTIAL FLAW: Display of data in web page
         * after using replaceAll() to remove script
         * tags, which will still allow XSS with strings
         * like <scr<script>ipt> (CWE 182: Collapse
         * of Data into Unsafe Value)
         */
        response.getWriter().println("<br>bad(): data = "
         + data.replaceAll("(<script>)", ""));
    }


}
```

BadSource-BadSink

Code vulnerabilities (Good vs. Bad)

- Benchmark - NIST testsuite
- SQL - example

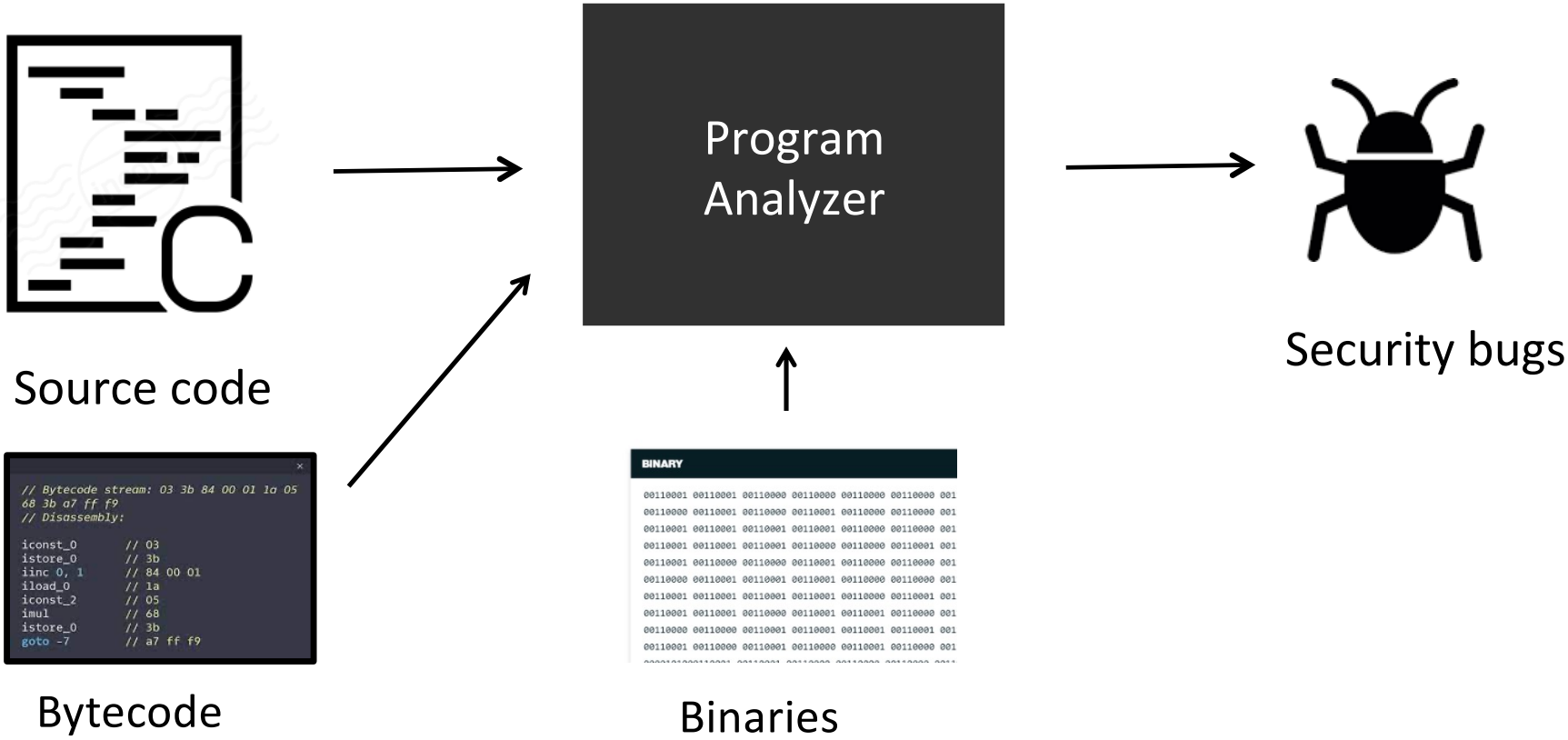
```
dbConnection = IO.getDBConnection();
sqlStatement = dbConnection.createStatement();
/* POTENTIAL FLAW: Read data using an outbound tcp connection
*/
data = readerBuffered.readLine();
/* POTENTIAL FLAW: data concatenated into SQL statement used
in executeQuery(), which could result in SQL Injection */
resultSet = sqlStatement.executeQuery("select * from users
where name='"+data+"'");
/* Use ResultSet in some way */
IO.writeLine(resultSet.getRow());
```

BadSource-BadSink

```
dbConnection = IO.getDBConnection();
sqlStatement = dbConnection.createStatement();
/* FIX: Use a hardcoded string */
data = "foo";
/* POTENTIAL FLAW: data concatenated into SQL statement
used in executeQuery(), which could result in SQL
Injection */
resultSet = sqlStatement.executeQuery("select * from
users where name='"+data+"'");
/* Use ResultSet in some way */
IO.writeLine(resultSet.getRow());
```

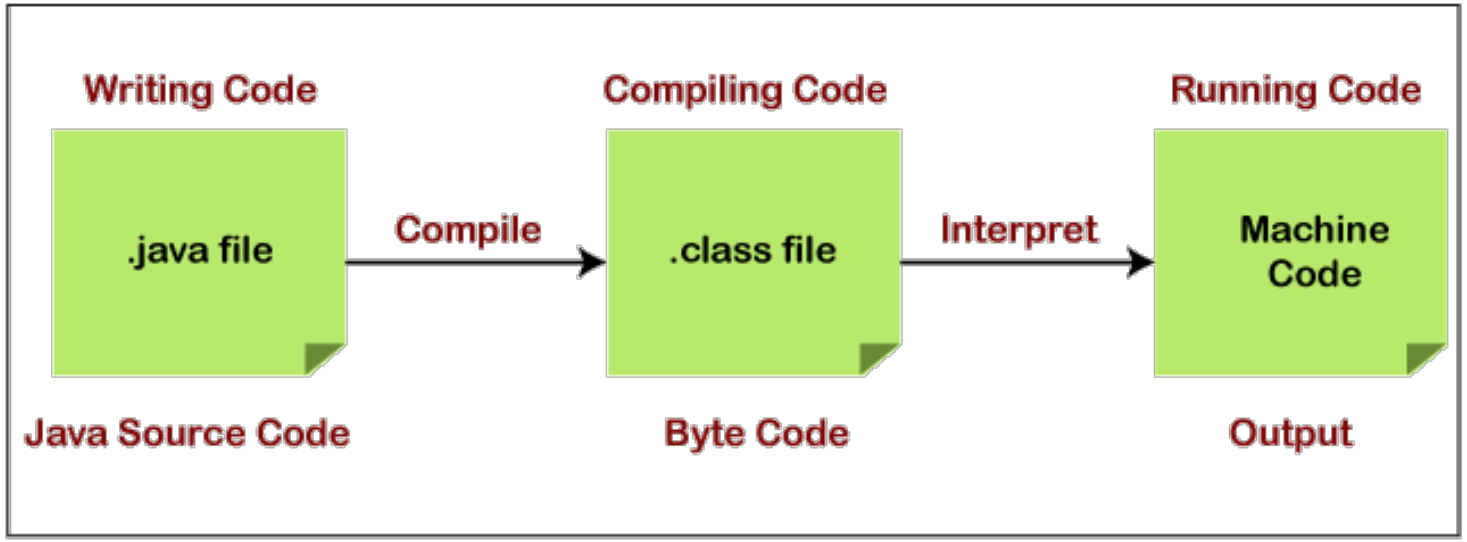
GoodSource-BadSink

Code representations



Byte code

.java – compiler – .class (bytecode)



```java
public class XmlDecodeUtil {


    public static Object handleXml(InputStream in) {
        XMLDecoder d = new XMLDecoder(in);
        try {
        //Deserialization happen here
            Object result = d.readObject();
            return result;
        }
        finally {
            d.close();
        }
    }
}
```

javap –p –c –constants XmlDecodeutil.class

```
public class spotbugs.XmlDecodeUtil {
  public spotbugs.XmlDecodeUtil();
    Code:
       0: aload_0
       1: invokespecial #8      // Method java/lang/Object."<init>":()V
       4: return


  public static java.lang.Object handleXml(java.io.InputStream);
    Code:
       0: new           #16      // class java/beans/XMLDecoder
       3: dup
       4: aload_0
       5: invokespecial #18      // Method java/beans/XMLDecoder."<init>":(Ljava/io/InputStream;)V
       8: astore_1
       9: aload_1
      10: invokevirtual #21      // Method java/beans/XMLDecoder.readObject:()Ljava/lang/Object;
      13: astore_2
      14: aload_2
      15: astore        4
      17: aload_1
      18: invokevirtual #25      // Method java/beans/XMLDecoder.close:()V
      21: aload         4
      23: areturn
      24: astore_3
      25: aload_1
      26: invokevirtual #25      // Method java/beans/XMLDecoder.close:()V
      29: aload_3
      30: athrow
    Exception table:
       from    to  target type
           9    17    24   any
}
```

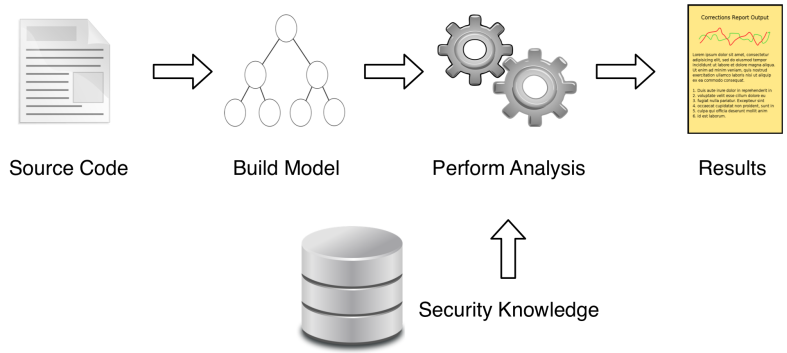
Bytecode instructions

Bytecode libraries: e.g.,

Apache BCEL: Apache Commons Bytecode Engineering Library, ASM:

Techniques for static code analysis (security)

- Model construction for source code analysis
  - Lexical Analysis
    - Split source/byte code into tokens to identify language constructs correctly. Remove unimportant tokens (e.g., whitespaces, comments, etc)
  - Semantic Analysis
    - Check the representation of each token for meaning (types, declarations, etc)
  - Control Flow Analysis
    - Possible paths that the program can take are determined and combined to several control flow graphs that represent all possible data flow paths
- Security knowledge



Techniques for static code analysis

- Pattern matching
- Control flow analysis
- Data flow analysis
- Taint analysis (Security)
  - Source (entry points)
  - Sink (Exit points)
  - Sanitization points (Filters)

Pattern Matching

- Report security issues if these patterns are found in the code
  - USE_OF_INSECURE_RANDOM_FUNCTION

Bug: This random generator (java.util.Random) is predictable
The use of a predictable random value can lead to vulnerabilities when used in certain security critical contexts. For example, when the value is used as:

✗ a CSRF token: a predictable token can lead to a CSRF attack as an attacker will know the value of the token

✗ a password reset token (sent by email): a predictable password token can lead to an account takeover, since an attacker will guess the URL of the "change password" form

✗ any other secret value

Bad Random Function - Example

```java
public class BadRandomFunction {

    public void generateRandomNumber() {
        Random r = new Random();
        r.nextInt();
    }
}
```

ByteCode

```java
if(clzNameInvoked.equals(BAD_RANDOM_FUNCTION)) {
    System.out.println("BAD RANDOM FUNCTION BUG Detected "
    + "\n@class: "+jclzName+"\n@Method: "+methodName
    +"\n@LineNumber: "+lineNumber);
}
```

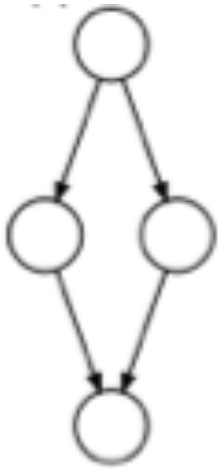
CodeAnalyzer

Report Bug

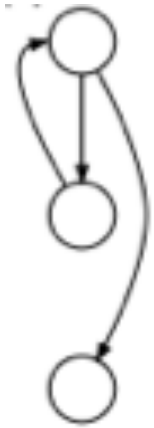
Control Flow

Code can have different control flows

- ✓ Control flow: If-Statements
- ✓ Control flow: While-Statements
- ✓ Control flow: For-Statements
- ✓ Control flow: Switch-Statements
- ✓ Control flow: Exceptions
- ✓ …



if-then-else



a while loop

Static analyzers need to cover flow branches during code analysis – Control Flow Graphs

Data Flow variants

- Different types
- data returned from one method to another in the same class
- data passed as an argument from one method to another in different classes in the same package
- …

Data Flow

Different patterns by which data flows in a program

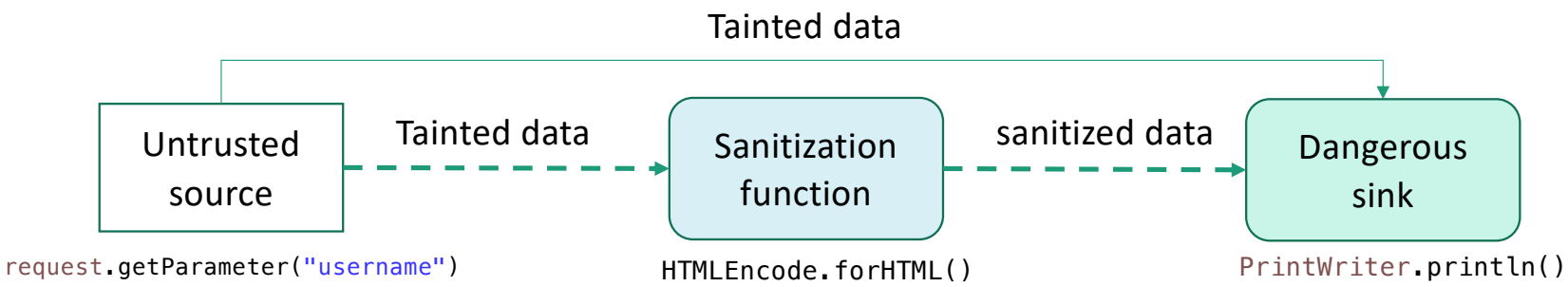
- ✓ Change in assignments of variables during the execution of a program
- ✓ Intra procedure (within method)
- ✓ Inter procedure (Between methods)
  - ✓ Within a class
  - ✓ Between classes
- ✓ Data flow uses the control flow graphs

```
public static Object handleXml() throws FileNotFoundException {
   String xmlfile = "Test.xml"; // hardcoded SAFE
   XMLDecoder d = new XMLDecoder(new BufferedInputStream(new FileInputStream(xmlfile)));
   try {
      Object result = d.readObject(); //Deserialization happen here
      return result;
    }
      finally {
         d.close();
      }
}
```

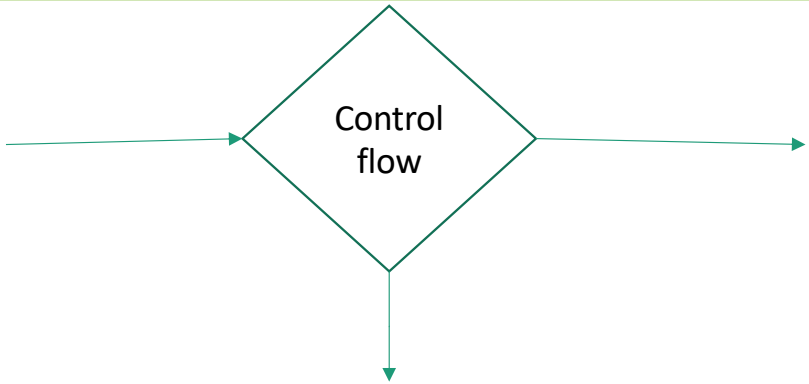
Intra procedure (within method)

Data Flow - Taint Analysis (Security)

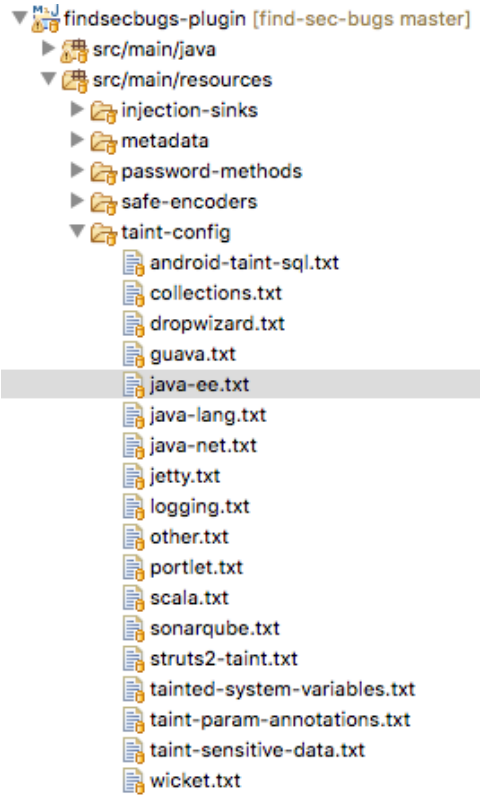
If the source of the input data is untrustworthy, then data is said to be tainted



If tainted data reaches a dangerous sink, a security issue may occur



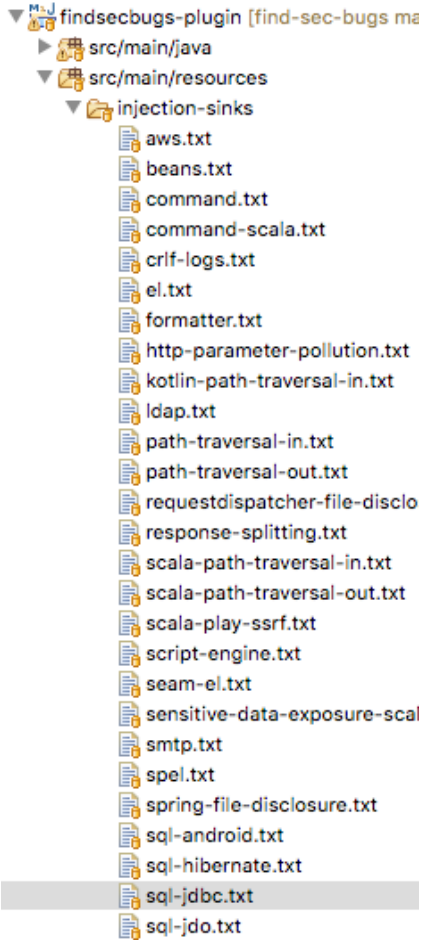
Taint Analysis – Sources (tainted)




```
javax/servlet/ServletRequest.getAttributeNames()Ljava/util/Enumeration;:SAFE
javax/servlet/ServletRequest.getCharacterEncoding()Ljava/lang/String;:SAFE
javax/servlet/ServletRequest.getContentType()Ljava/lang/String;:TAINTED
javax/servlet/ServletRequest.getLocalAddr()Ljava/lang/String;:TAINTED
javax/servlet/ServletRequest.getLocalName()Ljava/lang/String;:TAINTED
javax/servlet/ServletRequest.getParameter(Ljava/lang/String;)Ljava/lang/String;:TAINTED
javax/servlet/ServletRequest.getParameterMap()Ljava/util/Map;:TAINTED
```

A source is tagged tainted if its value can be manipulated by an external actor (process or human)

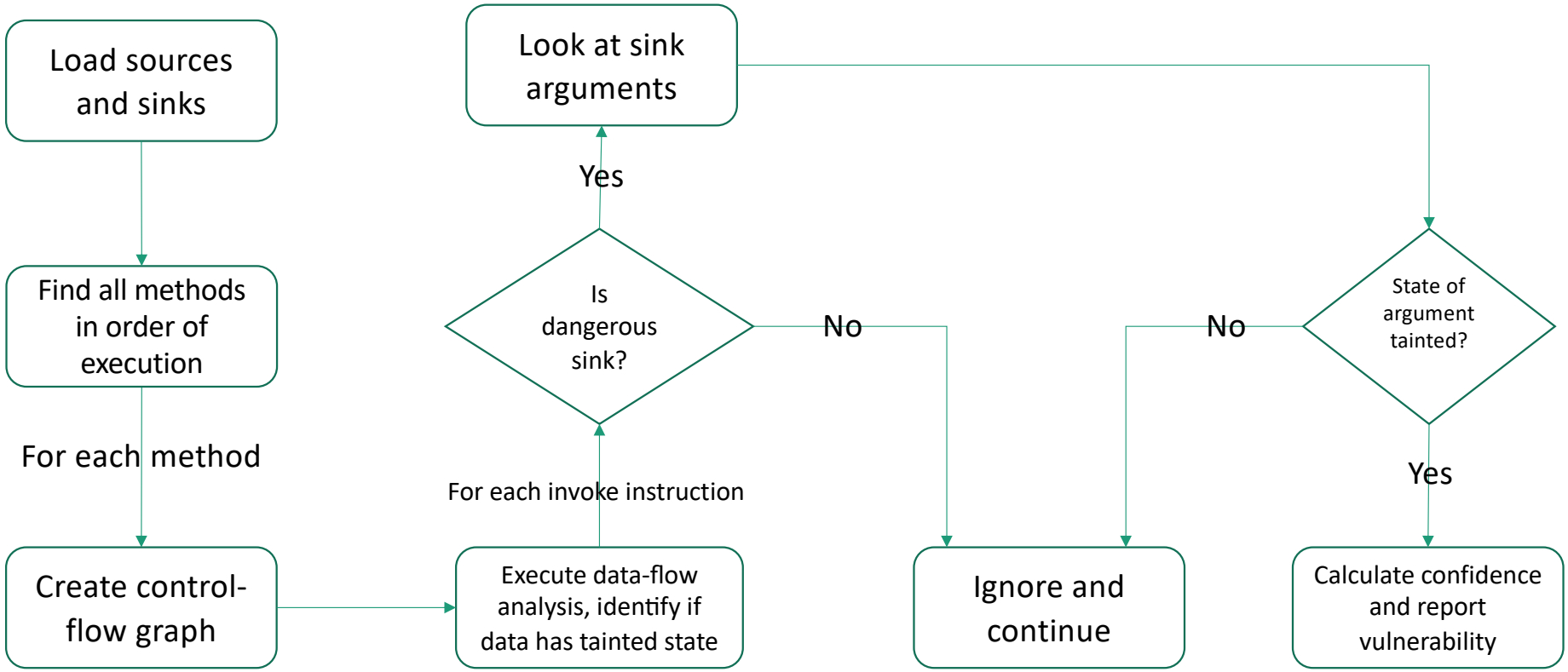
Taint Analysis – Sink (Dangerous)




```
java/sql/Statement.executeQuery(Ljava/lang/String;)Ljava/sql/ResultSet;:0
java/sql/Statement.execute(Ljava/lang/String;)Z:0
java/sql/Statement.execute(Ljava/lang/String;I)Z:1
java/sql/Statement.execute(Ljava/lang/String;[I)Z:1
java/sql/Statement.execute(Ljava/lang/String;[Ljava/lang/String;)Z:1
java/sql/Statement.executeUpdate(Ljava/lang/String;)I:0
java/sql/Statement.executeUpdate(Ljava/lang/String;I)I:1
java/sql/Statement.executeUpdate(Ljava/lang/String;[I)I:1
```

A sink is of security interest if its operation can read or write - CRUD

Taint analysis (e.g., FindSecBugs)



SQL Injection Analysis - FindSecBugs

- Detector Name:
  - SqlInjectionDetector
- Vulnerability Type:
  - SQL_INJECTION_JDBC
  - SQL_INJECTION_SPRING_JDBC
  - …
- Limitations:
  - Reports on all use of string concatenation in combination with a sink (exit point)
  - False positives when non-constant string is re-assigned a ‘safe’ value

SQL Injection Analysis - FindSecBugs

✗ Detector can't discriminate this SQL injection – False positive

```
/* POTENTIAL FLAW: data concatenated into SQL statement used in execute(), which could result in SQL Injection */
/* FIX: Use a hardcoded string */
data = "foo";
Boolean result = sqlStatement.execute("insert into users (status) values ('updated') where name='"+data+"'");
```

✓ Detector can detect a SQL injection based on tainted source

```
String data = ""; /* Initialize data */

InputStreamReader readerInputStream = new InputStreamReader(System.in, "UTF-8");
BufferedReader readerBuffered = new BufferedReader(readerInputStream);

/* POTENTIAL FLAW: Read data from the console using readLine */
data = readerBuffered.readLine();

/* POTENTIAL FLAW: data concatenated into SQL statement used in execute(), which could result in SQL Injection */
Boolean result = sqlStatement.execute(data);
```

Object and Context sensitivity

✓ Detector is both object and context insensitive

```
34          String name = request.getParameter("name");
35          UserNames un1 = new UserNames(name);              // vulnerable
36          UserNames un2 = new UserNames("Anonymous");       // not vulnerable
37
38          PrintWriter pw = response.getWriter();
39          pw.println(un1.getUser());
40          pw.println(un2.getUser());
```

Performance: How good are SAST tools?

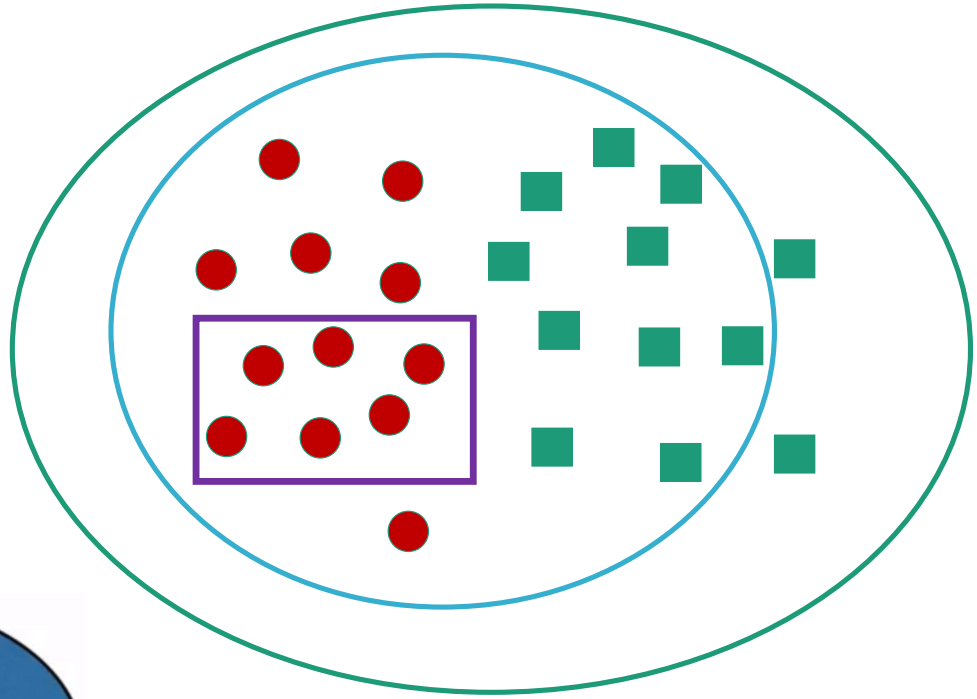
- We use certain metrics to measure how good a SAST tool perform
  - Soundness
  - Completeness
- True Positive
- True Negative
- False Positive
- False Negative

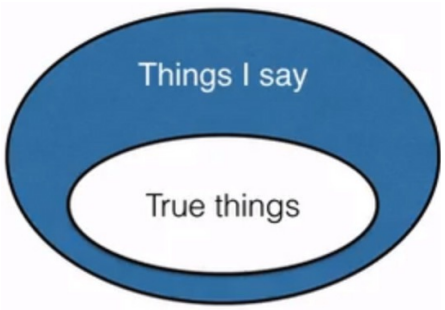
Soundness and Completeness

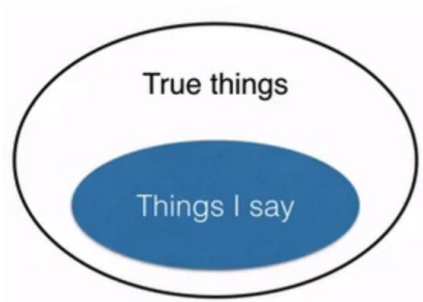
Non-security bug

Security bug

Total files = ■ + ●





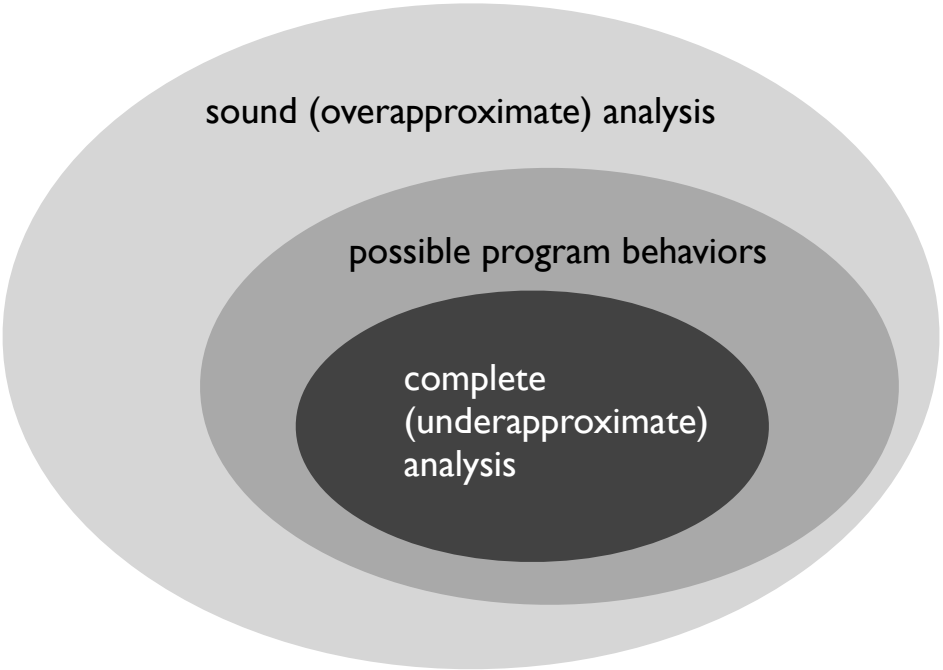


Soundness and Completeness

- **Soundness:** All reported bugs are real. **Sound** if it never overapproximates the set of bugs in a given program
  - If the analysis reports bugs that are not real bugs, then the analysis is **unsound**.
  - We refer to this situation as **false positives**
- **Completeness**: All bugs are reported. **Complete** if it never underapproximates the set of bugs in a given program.
  - If the analysis misses real bugs, then it is **incomplete**.
  - We refer to this misses as **false negatives**

Practical Static Analysis

- Practical static analysis tools' behaviour make compromises between soundness and completeness

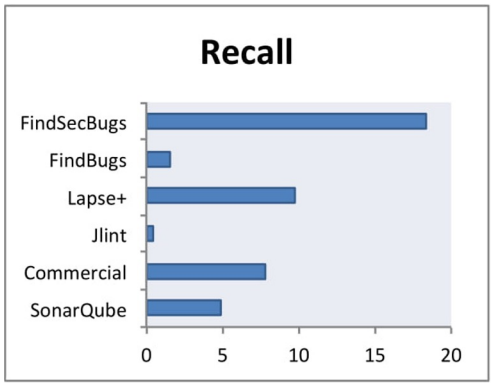


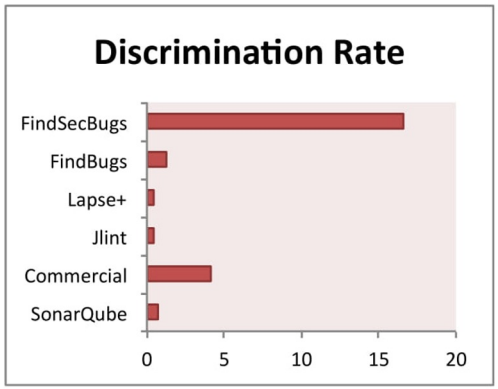
Test Suite – Performance of Tools

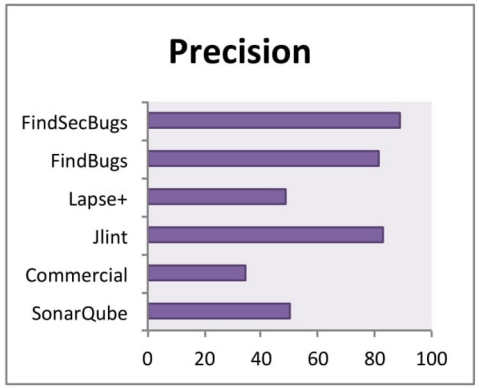
- Static analysis tools - overall performance

. Number of identified weaknesses by tools from a total of 26120 flaws

Tool	TP	FP	#Discrimination	Incidental flaws
SonarQube	1292	1275	200	237845
Commercial	2038	3834	1085	360212
FindSecBugs	4811	604	4338	41637
Lapse+	2550	2736	108	18950
JLint	125	26	104	586
FindBugs	426	98	341	22245







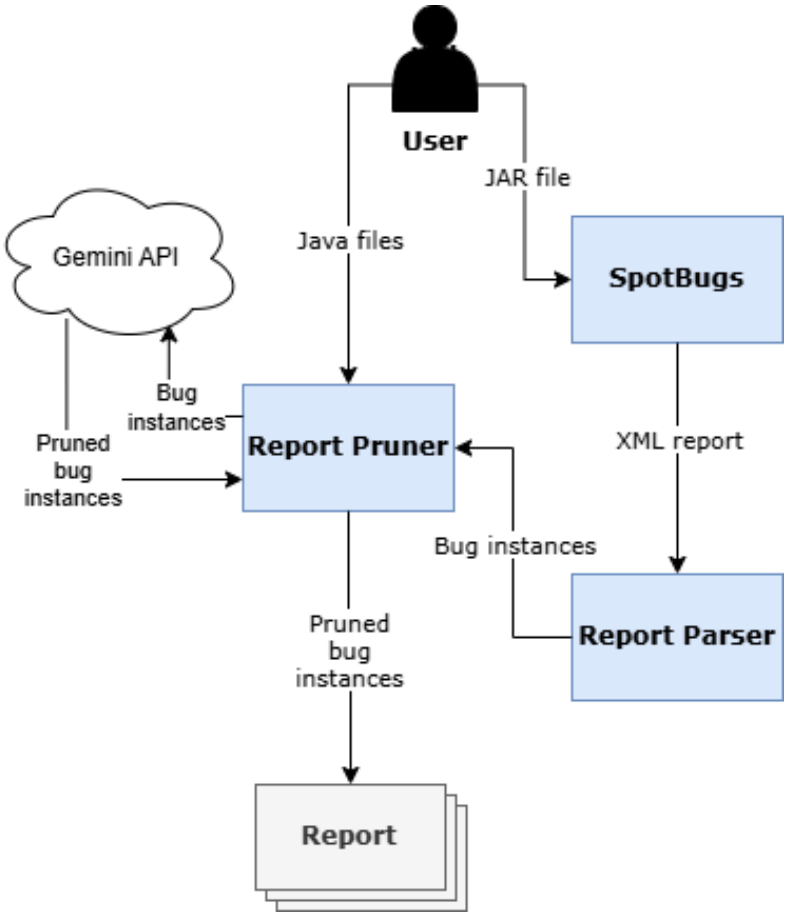
Recall = TP/TP+FN

Discrimination rate = #discriminations/TP+FN

Precision = TP/TP+FP

LLM-Assisted SAST

- Improve the SAST findings – reduce FPs
- Can also be used directly as SAST



- Svärd, J. and Teräs, M., 2025. Enhancing Code Static Analysis with LLMs: An Experimentwith SpotBugs and Gemini.
- Li, Z., Dutta, S. and Naik, M., 2024. IRIS: LLM-assisted static analysis for detecting security vulnerabilities. *arXiv preprint arXiv:2405.17238*.

LLM - SAST

```
34        String name = request.getParameter("name");
35        UserNames un1 = new UserNames(name);            // vulnerable
36        UserNames un2 = new UserNames("Anonymous");     // not vulnerable
37
38        PrintWriter pw = response.getWriter();
39        pw.println(un1.getUser());
40        pw.println(un2.getUser());
```

- Context sensitive

Prompt:
Analyse this file for vulnerability, mark the lines where vulnerability may occur and what types of vulnerability

GitHub Copilot

Findings for /servlets/ContextSensitive.java:

- Line: String name = request.getParameter("name");
- Issue: Untrusted input source. (tainted data)
- Risk: Any subsequent use of this value is dangerous unless validated/encoded.
- Line: UserNames un1 = new UserNames(name); // vulnerable
- Issue: Passing raw request data into an application object/constructor.
- Risk: Secondary injection depending on UserNames implementation (reflected XSS when later output, or SQL/LDAP/command injection if UserNames uses this value in a DB/OS/LDAP call).
- Line: response.getWriter().println(un1.getUser());
- Issue: Unsanitized user-supplied data written to HTTP response.
- Vulnerability: Reflected Cross-Site Scripting (XSS).
- Note: This is the primary sink — if getUser() returns the raw name, an attacker can inject script/HTML.
- **Line: response.getWriter().println(un2.getUser());**
- **Issue: Not vulnerable here because value is constant ("Anonymous"), but still an output sink — ensure future changes don't expose it.**

- Takk for i dag! ☺
- - og husk: de fleste angrep begynner i det små!